

1. What role does the Dockerfile play in containerization?

The Dockerfile acts as the blueprint or set of instructions for building a Docker image. It defines the base operating system, the necessary dependencies, environment variables, and the specific commands needed to set up and run an application. Without a Dockerfile, you would have to manually configure the environment every time you wanted to deploy the application, which would lead to inconsistencies across different systems.

2. Why are Docker containers lighter than virtual machines?

Docker containers are lighter because they share the host system's operating system (OS) kernel, whereas Virtual Machines (VMs) require a full, separate "guest" OS for each instance. Because containers don't need to load an entire OS to start, they use significantly less RAM and CPU, take up much less disk space (megabytes vs. gigabytes), and can start up in seconds rather than minutes.

3. What happens when the container finishes execution?

When the main process running inside a container (the one defined in the CMD or ENTRYPOINT of the Dockerfile) finishes or exits, the container stops running. It enters an "Exited" state. While the container is no longer active and consuming CPU/RAM, its file system state and metadata still exist on the host machine until the container is explicitly removed using a command like `docker rm`.